Date	
Team ID	
Project Name	Smart Metro Ticket Booking System
Maximum Marks	4 Marks

SMART METRO TICKET GENERATING SYSTEM

Technology Stack

1. Overview

The Smart Metro Ticket Generating System (SMTGS) is a distributed, cloud-native platform that enables commuters to plan journeys, purchase and validate tickets, and travel across the metro network with minimal friction. The technology stack is organized into four layers — Presentation, Application/API, Data, and Infrastructure — each chosen to meet the system's demanding requirements for high transaction throughput during peak commute hours, sub-second gate response times, strong payment security, and 24x7 availability across hundreds of stations.

This document describes the technologies selected for each layer, the rationale behind key choices, and how a single ticket-purchase request flows across the stack end to end.

1.1 Presentation Layer

The presentation layer provides four distinct entry points for commuters and staff:

- Mobile App (Flutter / React Native) — the primary channel for ticket purchase, smart-card recharge, live train tracking and digital wallet management, built cross-platform to minimize development effort across iOS and Android.
- Web Portal (React.js, Redux, TypeScript) — a browser-based alternative for desktop users and corporate/bulk-ticket buyers.
- Ticket Vending Machine (TVM) Kiosk UI (Angular, Electron) — a touchscreen kiosk application deployed at stations for cash and card payments.
- Station Gate / Validator UI — lightweight firmware-driven interface on turnstiles for tap-in/tap-out interactions.

1.2 Application / API Layer

All client applications communicate through an API Gateway (Kong/NGINX) that centralizes authentication, rate limiting and TLS termination before requests reach domain-specific microservices: the Ticketing Microservice (fare rules, ticket issuance), the Payment Service (UPI/NFC/card orchestration), and the Auth & Fare Calculation Service (OAuth2.0/JWT-based identity and dynamic distance-based fare computation).

2. Data, Infrastructure & Full Technology Matrix

The Data layer combines a relational store for transactional integrity, an in-memory cache for latency-sensitive fare lookups, and an event streaming backbone that decouples services for resilience. The Infrastructure layer provides elastic compute via a managed Kubernetes cluster on a public cloud, IoT hardware at station gates, and a security envelope spanning encryption, web application firewalls and identity management.

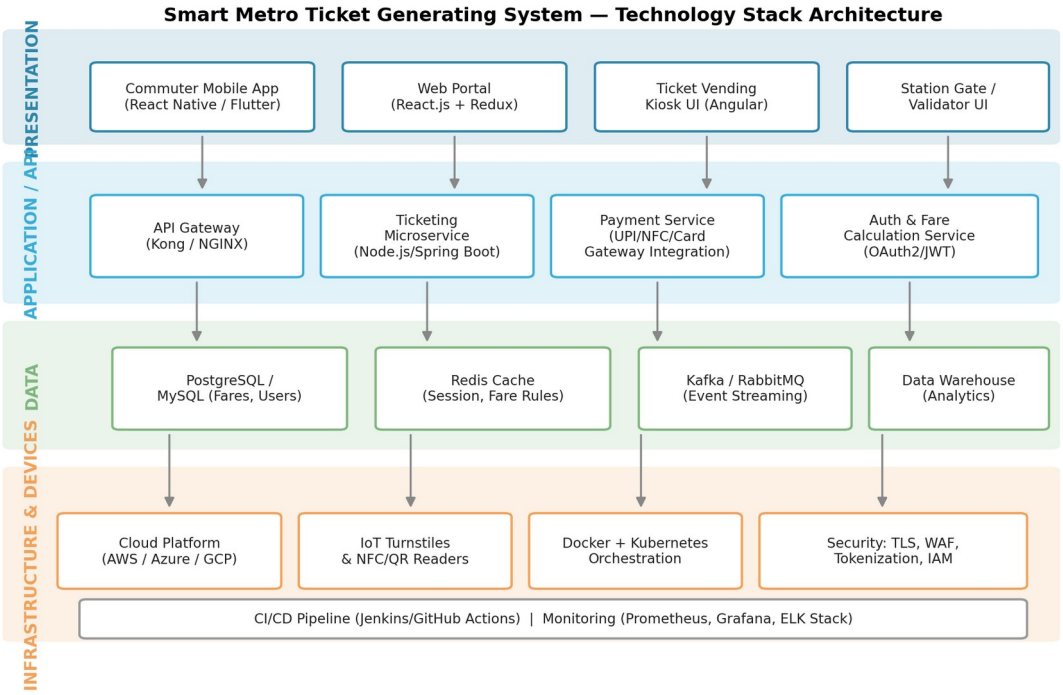

Figure 1: Layered technology stack architecture for the Smart Metro Ticket Generating System

2.1 Complete Technology Matrix

Layer	Technology / Tool	Purpose
Mobile App	Flutter / React Native	Cross-platform commuter app for iOS & Android
Web Portal	React.js, Redux, TypeScript	Self-service ticketing and account management
Kiosk / TVM UI	Angular, Electron	Touchscreen ticket vending machine interface
API Gateway	Kong / NGINX	Request routing, rate limiting, TLS termination
Ticketing Service	Node.js / Spring Boot	Core fare and ticket issuance logic
Payment Service	Java / Node.js + UPI, NFC, Card SDKs	Payment orchestration and reconciliation
Auth Service	OAuth2.0, JWT, Keycloak	Identity, session and token management
Relational DB	PostgreSQL / MySQL	Transactional data: users, fares, tickets
Cache	Redis	Low-latency fare rule and session lookups
Event Streaming	Apache Kafka / RabbitMQ	Asynchronous events between microservices
Analytics Store	Data Warehouse (Snowflake/BigQuery)	Reporting, revenue and ridership analytics
Cloud Platform	AWS / Azure / GCP	Hosting, autoscaling, managed services
IoT Devices	NFC/QR Readers, Turnstile Controllers	Physical gate entry and exit validation
Container Platform	Docker, Kubernetes	Service packaging and orchestration
Security	TLS 1.3, WAF, IAM, Tokenization	Data-in-transit/at-rest protection, access control
CI/CD	Jenkins / GitHub Actions	Automated build, test and deployment pipelines
Monitoring	Prometheus, Grafana, ELK Stack	Observability, logging and alerting

3. Ticket Request Flow Across the Stack

The diagram below traces a single ticket-purchase transaction as it moves through every layer of the stack — from the commuter's device, through authentication and fare lookup, to payment authorization, persistence, and final QR-code delivery. This flow illustrates how the chosen technologies interoperate in production and highlights the synchronous (API) versus asynchronous (event-driven) paths used to keep the system responsive under load.

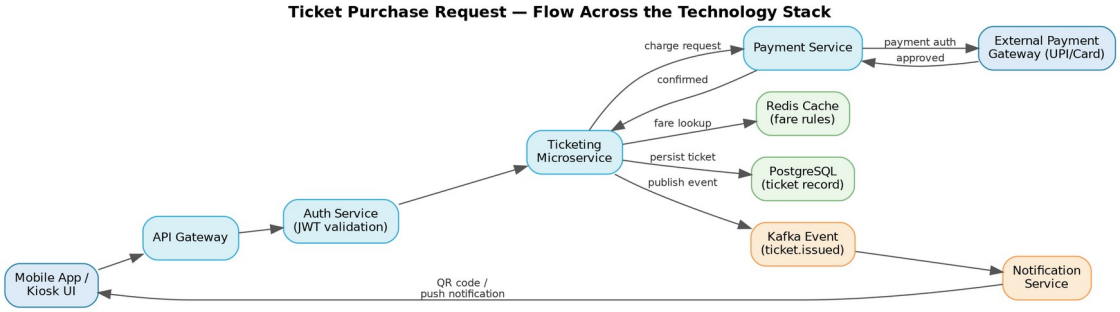

Figure 2: End-to-end flow of a ticket purchase request across the technology stack

3.1 Design Considerations

- Statelessness: application services are stateless so that Kubernetes can horizontally autoscale during morning/evening peak ridership.
- Caching strategy: fare rules and station metadata are cached in Redis with short TTLs to keep gate-side validation under 200ms.
- Event-driven decoupling: ticket issuance publishes a Kafka event consumed independently by notification, analytics and fraud-detection services, so a slow downstream consumer never blocks ticket delivery.
- Security by design: all payment data is tokenized at the Payment Service boundary; PAN data never touches the ticketing or analytics stores, supporting PCI-DSS compliance.
- Observability: every service emits structured logs and metrics to the ELK stack and Prometheus/Grafana, enabling real-time dashboards for station-level ticketing volume and system health.

3.2 Summary

Together, these choices give the Smart Metro Ticket Generating System a stack that is horizontally scalable, secure by default, and resilient to the failure of any single downstream dependency — while still delivering the near-instant gate response commuters expect during rush hour.